Module 14: Network Architecture

Scalable Distributed System Design

System Architecture Overview

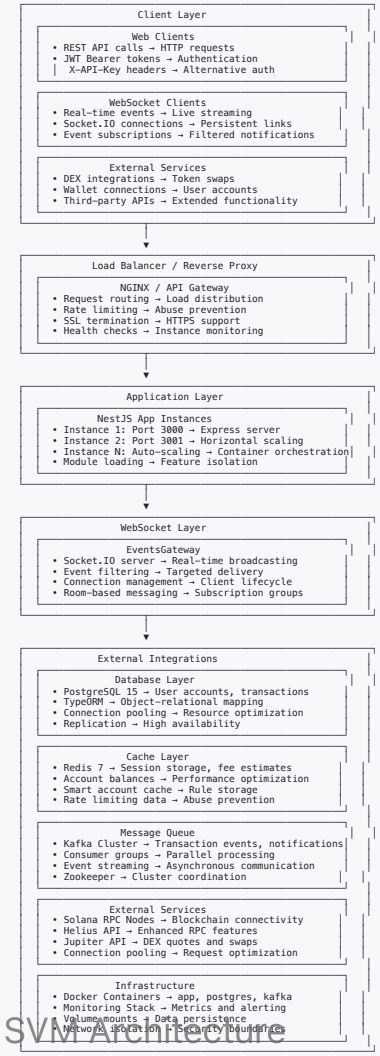
Architecture Principles

- **Microservices Design**: Modular, independently deployable services
- **Horizontal Scaling**: Multiple application instances behind load balancer
- **Event-Driven Communication**: Asynchronous messaging with Kafka
- **Multi-Layer Caching**: Redis for performance optimization
- **External API Integration**: Solana RPC and DEX service connections

Scalability Features

- **Load Balancing**: Request distribution across multiple instances
- **Database Connection Pooling**: Efficient database resource management
- **WebSocket Broadcasting**: Real-time event distribution
- **Message Queuing**: Asynchronous processing and decoupling

System Architecture



Load Balancing & Scaling

NGINX Configuration

```
# nginx.conf
upstream backend {
    least_conn;
    server app:3000;
    server app:3001;
    server app:3002;
}

server {
    listen 80;
    server_name api.solana-svm-study.com;

    # Rate limiting
    limit_req zone=api burst=10 nodelay;

    # SSL termination
    listen 443 ssl http2;
    ssl_certificate /etc/ssl/certs/api.crt;
    ssl_certificate_key /etc/ssl/private/api.key;

    # API routing
    location /api/ {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # WebSocket routing
    location /socket.io/ {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Health checks
    location /health {
        access_log off;
```

Database Architecture

PostgreSQL Configuration

```
// Connection pooling with TypeORM
export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT),
  username: process.env.DB_USERNAME,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,

  // Connection pool settings
  extra: {
    max: 20,              // Maximum connections
    min: 5,               // Minimum connections
    idleTimeoutMillis: 30000,
    acquireTimeoutMillis: 60000,
  },

  // Replication for read scaling
  replication: {
    master: {
      host: process.env.DB_MASTER_HOST,
      port: parseInt(process.env.DB_MASTER_PORT),
      username: process.env.DB_MASTER_USER,
      password: process.env.DB_MASTER_PASSWORD,
    },
    slaves: [{
      host: process.env.DB_SLAVE_HOST,
      port: parseInt(process.env.DB_SLAVE_PORT),
      username: process.env.DB_SLAVE_USER,
      password: process.env.DB_SLAVE_PASSWORD,
    }],
  },
  synchronize: false,
  logging: process.env.NODE_ENV === 'development',
```

Caching Strategy

Redis Implementation

```
// Redis service for multi-purpose caching
@Injectable()
export class RedisService {
  constructor(private readonly redis: Redis) {}

  // Session storage
  async setSession(sessionId: string, data: any, ttl: number = 3600): Promise<void> {
    await this.redis.setex(`session:${sessionId}`, ttl, JSON.stringify(data));
  }

  async getSession(sessionId: string): Promise<any> {
    const data = await this.redis.get(`session:${sessionId}`);
    return data ? JSON.parse(data) : null;
  }

  // Fee estimation cache
  async cacheFeeEstimate(txSignature: string, estimate: FeeEstimate): Promise<void> {
    await this.redis.setex(`fee:${txSignature}`, 300, JSON.stringify(estimate)); // 5 min TTL
  }

  async getCachedFeeEstimate(txSignature: string): Promise<FeeEstimate | null> {
    const data = await this.redis.get(`fee:${txSignature}`);
    return data ? JSON.parse(data) : null;
  }

  // Account balance cache
  async cacheAccountBalance(address: string, balance: number): Promise<void> {
    await this.redis.setex(`balance:${address}`, 60, balance.toString()); // 1 min TTL
  }

  async getCachedAccountBalance(address: string): Promise<number | null> {
    const balance = await this.redis.get(`balance:${address}`);
    return balance ? parseInt(balance) : null;
  }

  // Rate limiting
  async checkRateLimit(key: string, limit: number, window: number): Promise<boolean> {
    const count = await this.redis.incr(key);
    if (count === 1) {
      await this.redis.expire(key, window);
    }
```

Message Queue Architecture

Kafka Configuration

```
// Kafka producer configuration
export const kafkaConfig = {
  clientId: 'solana-svm-study',
  brokers: ['kafka:9092'],
  retry: {
    initialRetryTime: 100,
    retries: 8,
  },
};

// Producer service
@Injectable()
export class KafkaProducerService {
  private producer: Producer;

  async onModuleInit() {
    this.producer = this.kafka.producer();
    await this.producer.connect();
  }

  async publishTransactionEvent(event: TransactionEvent): Promise<void> {
    await this.producer.send({
      topic: 'transaction-events',
      messages: [{
        key: event.signature,
        value: JSON.stringify(event),
      }],
    });
  }

  async publishAccountEvent(event: AccountEvent): Promise<void> {
    await this.producer.send({
      topic: 'account-events',
      messages: [{
        key: event.address,
        value: JSON.stringify(event),
      }],
```

WebSocket Architecture

Real-Time Event Broadcasting

```
@WebSocketGateway({
  namespace: '/events',
  cors: { origin: '*' },
  transports: ['websocket', 'polling'],
})
export class EventsGateway implements OnGatewayConnection, OnGatewayDisconnect {
  @WebSocketServer()
  server: Server;

  private clients = new Map<string, Socket>();

  handleConnection(client: Socket): void {
    this.clients.set(client.id, client);
    client.join(`client:${client.id}`);
    console.log(`Client connected: ${client.id}`);
  }

  handleDisconnect(client: Socket): void {
    this.clients.delete(client.id);
    console.log(`Client disconnected: ${client.id}`);
  }

  @SubscribeMessage('subscribe')
  handleSubscribe(client: Socket, payload: SubscribePayload): void {
    const { eventTypes, filters } = payload;

    // Create subscription
    const subscription = this.subscriptionService.createSubscription({
      clientId: client.id,
      eventTypes,
      filters,
    });

    // Join subscription room
    client.join(`subscription:${subscription.id}`);

    // Send confirmation
    client.emit('subscribed', { subscriptionId: subscription.id });
  }

  @SubscribeMessage('unsubscribe')
  handleUnsubscribe(client: Socket, payload: { subscriptionId: string }): void {
    client.leave(`subscription:${payload.subscriptionId}`);
    this.subscriptionService.deleteSubscription(payload.subscriptionId);
  }

  async broadcastEvent(event: Event): Promise<void> {
    // Get active subscriptions for this event type
    const subscriptions = await this.subscriptionService.getSubscriptionsByEventType(
      event.eventType
    );

    // Broadcast to matching subscriptions
    for (const subscription of subscriptions) {
      if (this.matchesFilters(event, subscription.filters)) {
```

External Service Integration

Solana RPC Connection Pooling

```
@Injectable()
export class SolanaRpcService {
  private connections: Connection[] = [];

  constructor() {
    // Initialize connection pool
    const rpcUrls = [
      'https://api.mainnet-beta.solana.com',
      'https://solana-api.projectserum.com',
      'https://rpc.ankr.com/solana',
    ];

    for (const url of rpcUrls) {
      this.connections.push(new Connection(url, 'confirmed'));
    }
  }

  // Round-robin connection selection
  private getConnection(): Connection {
    const index = Math.floor(Math.random() * this.connections.length);
    return this.connections[index];
  }

  async getAccountInfo(address: PublicKey): Promise<AccountInfo<Buffer> | null> {
    const connection = this.getConnection();
    return await connection.getAccountInfo(address);
  }

  async getBalance(address: PublicKey): Promise<number> {
    const connection = this.getConnection();
    return await connection.getBalance(address);
  }
  async getRecentBlockhash(): Promise<{ blockhash: string; lastValidBlockHeight: number }> {
    const connection = this.getConnection();
```

Monitoring & Observability

Application Metrics

```
// Prometheus metrics
import { register, collectDefaultMetrics, Gauge, Counter, Histogram } from 'prom-client';

// Enable default metrics
collectDefaultMetrics();

// Custom metrics
export const httpRequestDuration = new Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
});

export const activeConnections = new Gauge({
  name: 'websocket_active_connections',
  help: 'Number of active WebSocket connections',
});

export const databaseConnections = new Gauge({
  name: 'database_active_connections',
  help: 'Number of active database connections',
});

export const kafkaMessagesProcessed = new Counter({
  name: 'kafka_messages_processed_total',
  help: 'Total number of Kafka messages processed',
```

Docker & Container Orchestration

Production Docker Compose

```yaml
version: '3.8'
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000-3002:3000"  # Multiple ports for scaling
    environment:
      - NODE_ENV=production
      - DB_HOST=postgres
      - REDIS_HOST=redis
      - KAFKA_BROKER=kafka:9092
    depends_on:
      - postgres
      - redis
      - kafka
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure

  postgres:
    image: postgres:15-alpine
    environment:
      - POSTGRES_DB=solana_svm
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    deploy:
      resources:
        limits:
          memory: 1G
        reservations:
          memory: 512M

  redis:
    image: redis:7-alpine
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data
    deploy:
      resources:
        limits:
          memory: 512M

  kafka:
    image: confluentinc/cp-kafka:7.3.0
    environment:
      - KAFKA_BROKER_ID=1
      - KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181
      - KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
      - KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1
    depends_on:
      - zookeeper
    volumes:
      - kafka_data:/var/lib/kafka/data

  zookeeper:
    image: confluentinc/cp-zookeeper:7.3.0
    environment:
      - ZOOKEEPER_CLIENT_PORT=2181

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./ssl:/etc/ssl/certs
    depends_on:
```

Key Takeaways

Architecture Benefits

- **Scalability**: Horizontal scaling with load balancing
- **Reliability**: Multiple instances prevent single points of failure
- **Performance**: Caching, connection pooling, and async processing
- **Maintainability**: Modular design with clear separation of concerns
- **Observability**: Comprehensive monitoring and health checks

Production-Ready Features

- **Container Orchestration**: Docker-based deployment and scaling
- **Database Optimization**: Connection pooling and read replicas
- **Message Queuing**: Asynchronous processing with Kafka
- **Real-Time Communication**: WebSocket broadcasting for live updates